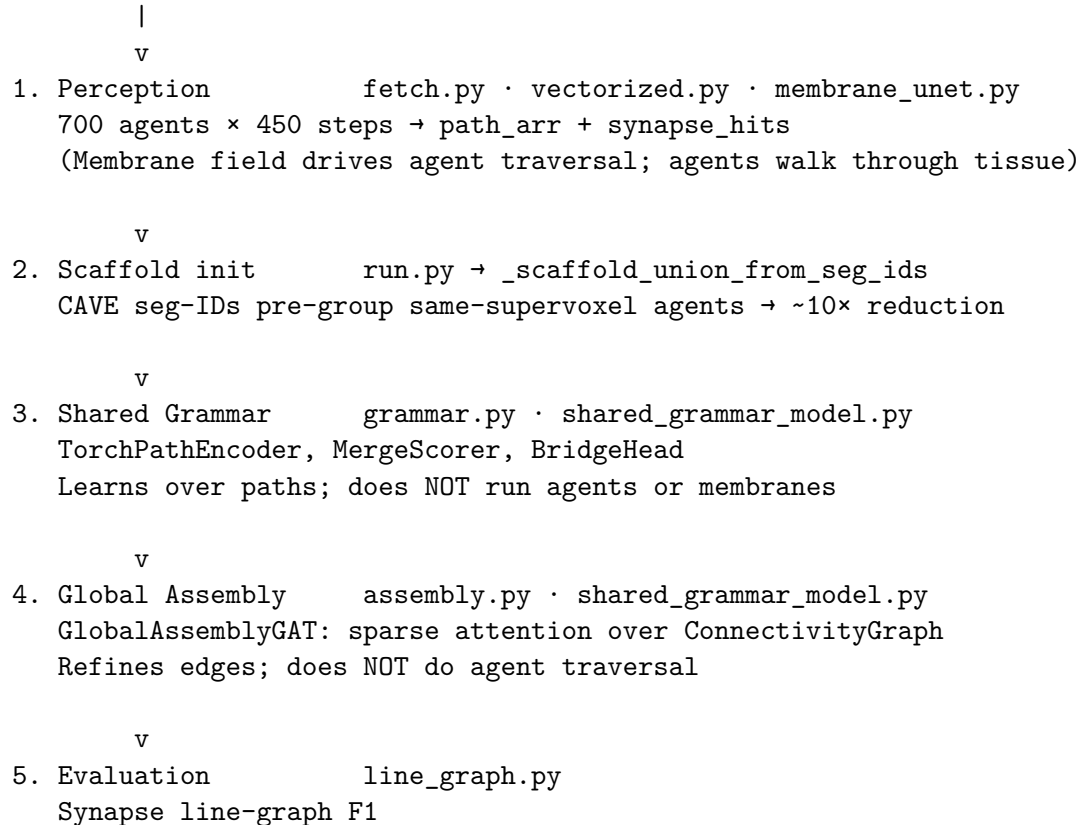


Neuronauts Model & Architecture Note

Captured from design discussion. Covers: agent/grammar/GAT pipeline, global vs box-scale, and the question of hierarchical assembly.

Architecture Overview

EM volume + CAVE synapses



What Is an Agent?

An **agent** is one of the 700 virtual walkers that trace paths through the EM volume.

- **Agent** = one path/trace. Each has: `agent_id` (0–699), `path` (sequence of 3D voxel points), `visited_synapses`.
- Agents walk in the membrane field (repelled by membranes, attracted to synapses).
- One agent ~ one neurite trace. Many agents can represent one neuron after merging.

Pipeline mapping:

1. 700 agents → `path_arr` [700, 450, 3], `synapse_hits` [700, `n_syn`]
2. Scaffold init groups agents by CAVE seg-ID
3. Merge + beam search: Grammar MergeScorer decides which agent-pairs to merge
4. **MergedNeuron** = 1+ agents merged together. Has `path_points`, `synapse_indices`, `agent_ids`

5. **ConnectivityGraph** = nodes = MergedNeurons, edges = synapse connections
6. **GAT** sees MergedNeurons (not raw agents). Node features = path_encoder(path_points). Refines which synapse edges to keep.

The GAT does not run agents. It operates on the graph of fragments and synapse edges after merge.

Grammar vs GAT

Component	Role	Uses EM/membranes?	Uses agents?
Perception	Membrane field + agent traversal	Yes	Yes
Grammar	Path encoding, merge scoring, atomicity, bridge	No (paths derived from agents)	Indirect (paths from agents)
GAT	Global graph refinement	No	No (sees MergedNeurons)

Grammar training can run **without agent simulation** — it uses cached synapse tables for merge/atomicity. GAT training needs agent simulation to build ConnectivityGraphs.

Grammar Embeddings as Node Features

The GAT consumes **node features** produced by the shared path encoder. Here is the full flow from raw path points to embeddings.

1. Path points to feature sequence

Each MergedNeuron has path_points — a (K, 3) array of 3D voxel coordinates along the neurite trace. These are converted to a **per-step feature sequence** [T, 3] where T = K-1:

path_points [K, 3] --> _path_seq_from_pts --> seq [T, 3]

Per-step features (in physical nm, Z scaled for anisotropy):

- **edge_len** — length of each segment
- **radius** — distance from path centroid
- **curvature** — cumulative turning angle (from unit tangent changes)

Heuristic vs learned: These per-step features are **heuristic** — hand-designed geometric descriptors. The choice of (edge_len, radius, curvature), their formulas, and the Z-scaling are all fixed by hand. The **Torch-PathEncoder** is what is learned: it aggregates this heuristic sequence into embeddings for merge scoring,

atomicity, bridge prediction, and GAT node features. So the *input representation* is heuristic; the *aggregation and downstream use* are learned.

Defined in `assembly.py` `_path_seq_from_pts`; matches `run.py` `_path_sequence_from_points` so the encoder sees the same geometry as the merge scorer.

2. TorchPathEncoder forward pass

The encoder (`grammar.py` `TorchPathEncoder`) processes batched sequences:

```
Input:  x [B, T, 3],  mask [B, T]  (True = PAD)
|
v
input_proj:  [B, T, 3] -> [B, T, d_model]
|
v
Prepend [CLS] token, add sinusoidal positional encodings
|
v
TransformerEncoder (2 layers, 4 heads, d_model=64)
|
v
Take [CLS] output at position 0 --> output_proj --> [B, output_dim]

• output_dim = 32 (configurable)
• max_len = 512 — long paths are truncated to stay within the positional budget
• The [CLS] token aggregates the full sequence via attention; no manual pooling
```

3. Box-level GAT usage

In `assembly.py` `_encode_neurons`:

- One `MergedNeuron` → one path → one embedding `h` [`embedding_dim`]
- Neurons with no path points get an all-zero embedding
- GAT receives `h` [`N`, `embedding_dim`] as node features for the box `ConnectivityGraph`

4. Soma graph usage (planned)

For the **soma graph** (`experiments/soma_graph/`), nodes are *neurons* (root IDs), not fragments. Each neuron may have multiple fragments across overlapping boxes or multiple traces:

- Run agents in a neighborhood box per neuron (or use cached fragments)
- Encode each fragment path with `TorchPathEncoder`
- **Pool** per neuron: mean or max over fragment embeddings → one [`embedding_dim`] vector per root
- Use these as soma graph node features

Currently `build_soma_graph_from_synapses` uses placeholder features (random or zeros). Replacing them with grammar-pooled embeddings is the intended production path.

5. Fully learned path representation (no heuristics)

To eliminate heuristics entirely, feed **raw path geometry** into the encoder and let it learn its own abstractions:

- **Raw step vectors** — $(\Delta x, \Delta y, \Delta z)$ or $(\Delta x, \Delta y, \Delta z) \times \text{voxel_size_nm}$ per segment. The encoder learns which geometric properties matter for merge/atomicity.
- **Raw positions** — (x, y, z) per vertex with positional encoding along the path. The model infers edge length, curvature, etc. from coordinates.
- **Point cloud encoder** — If local EM context is available, treat the path as a point cloud and use PointNet, PointTransformer, or a voxel CNN over a local neighborhood. Geometry is learned from voxel intensities, not precomputed descriptors.

Tradeoffs: A fully learned input avoids bias from human-chosen features and can discover structure we didn't anticipate. Costs: less interpretability, more data and compute, and potential sensitivity to coordinate frame or voxel resolution. The current heuristic input is cheap, interpretable, and already effective; moving to raw inputs is a natural next step if we hit a ceiling.

Global vs Box-Scale

Current pipeline: Box-level (6–30 μm). One box $\rightarrow$ one ConnectivityGraph $\rightarrow$ one GAT pass.

Minnie65 scale: $\sim 120\text{k}$ neurons, $\sim 300\text{M}$ synapses, $\sim 1\text{ mm}^3$. “All at once” is infeasible.

Decomposition options:

1. **Per-nucleus** — One subgraph per neuron (soma position + synapse cloud). Embarrassingly parallel.
 2. **Tiling** — Overlapping 20–40 μm tiles, run agents + merge per tile, stitch at boundaries.
 3. **Soma graph** — Neuron $\times$ neuron graph (120k nodes, sparse edges). GAT over neurons, not fragments. See `experiments/soma_graph/`.
-

Do We Need a Hierarchical Global Assembly Step?

Short answer: Not for the current box-level pipeline. Yes if we want whole-dataset inference.

Current design: GAT is “global” only within a single box — it sees all fragments and edges in that box. For 50–200 nodes per box, a single GPU is sufficient.

Hierarchical step would mean:

- **Level 1 (current):** Box-level merge + GAT $\rightarrow$ refined ConnectivityGraph per box
- **Level 2 (proposed):** Cross-box stitching / refinement
 - Option A: Fuse overlapping box boundaries (same fragments appear in multiple tiles)
 - Option B: Soma-level graph — aggregate box outputs into neuron $\times$ neuron, run GAT at that scale
 - Option C: Coarse-to-fine — cheap heuristics on 100 μm to propose clusters, refine with agents+GAT on smaller subvolumes

When to add it: When we move beyond single-box evaluation (e.g. soma graph experiment, multi-box validation, or full Minnie65 inference). The current pipeline is complete for box-scale; hierarchical assembly is the bridge to global.

Iterative Optimization (Codex/Gemini)

- `scripts/codex_optimize.py` — Patch → evaluate → keep/revert loop
- `program.md` — Mission + architecture; context for the LLM
- **Default target:** `neuronauts/grammar.py`
- **Validation:** Research cycle → `val_f1`; keep changes that improve it
- **Backends:** Codex, Claude, Gemini

The LLM modifies code to improve validation F1 without hard-coded heuristics. The LLM patches only `grammar.py`; the GAT and assembly pipeline run during validation but are not modified. The learned object is the grammar; the LLM is an outer optimizer.

Morphology vs Connectivity

Current supervision: Root IDs (same-root = same neuron). Implicit structure, no explicit morphology model.

Possible addition: Morphology / validity head trained on CAVE segments — “does this cluster look like a valid neuron?” Could use pooled path features per root as a prior or diagnostic.

Program.md: Richer path features (skeleton tortuosity, mesh volume-surface ratio) noted as future improvement.

Key Files

File	Purpose
<code>neuronauts/vectorized.py</code>	Agent simulation
<code>neuronauts/merge.py</code>	MergedNeuron, <code>merge_agents</code>
<code>neuronauts/run.py</code>	Full pipeline, <code>_build_graph</code>
<code>neuronauts/assembly.py</code>	<code>gat_refine_connectivity</code> , <code>label_graph_edges</code>
<code>neuronauts/shared_grammar_model.py</code>	<code>GlobalAssemblyGAT</code> , <code>gat_train_step</code>
<code>experiments/soma_graph/</code>	Soma-level neuron × neuron experiment

Generated from design discussion. Last updated: 2025-03.